

IPC 2026 — Pràctiques

Running la Safor

Interfícies Persona Computador · IPC · UPV

Aquest document descriu el cas d'estudi que heu d'implementar com a projecte de l'assignatura.

La biblioteca gestiona tots els aspectes tècnics — centreu-vos en el disseny de la interfície i la interacció.

Contenido

1. Introducció	4
2. Cas d'estudi	4
Categories d'escenaris	4
3. Escenaris sobre l'usuari	5
3.1 Registrar-se a l'aplicació	5
3.2 Autenticar-se	5
3.3 Modificar perfil	5
3.4 Tancar sessió	6
3.5 Visualitzar l'historial de sessions	6
4. Escenaris sobre les activitats	6
4.1 Registrar una activitat nova	6
4.2 Afegir anotacions a l'activitat	6
4.3 Visualitzar una activitat	6
4.4 Acumulat d'activitats	6
5. Escenaris sobre la ruta en el mapa	6
5.1 Realitzar zoom	6
6. Escenaris addicionals	7
6.1 Perfil de desnivell	7
6.2 Velocitat sobre el traçat	7
7. Escenaris sobre mapes	7
7.1 Afegir un mapa al sistema	7
8. Recursos per a realitzar la pràctica	7
8.1 Els mapes	8
8.2 Els fitxers GPX	9
8.3 Model de dades i classes de la biblioteca	9
User	10
Mètodes estàtics de validació	10
Session	11
Activity	11
TrackPoint	12
Annotation	12
AnnotationType	13
GeoPoint	13
MapRegion	13
MapProjection	14
GeoUtils	15
8.4 La classe SportActivityApp	15

Gestió d'usuaris	15
Gestió d'activitats	16
Gestió d'anotacions	16
Gestió de mapes	16
Sessions.....	17
9. Exemple d'ús i estructura del projecte	17
9.1 Exemple d'ús	17
9.2 Estructura del projecte a desenvolupar	18
9.3 Afegir les biblioteques a NetBeans	18
10. Persistència de les dades	19
11. Projecte base	19
12. Instruccions de lliurament	19

1. Introducció

Aquest document descriu el cas d'estudi que heu d'implementar com a projecte de l'assignatura. Per centrar el treball en el disseny de la interfície i la interacció amb l'usuari, es proporciona una biblioteca (IPC2026.jar) que s'encarrega de tota la part tècnica: model de dades, persistència en base de dades SQLite, anàlisi de fitxers GPX i càlculs geogràfics.

El document s'organitza de la manera següent:

- Cas d'estudi: descripció del sistema a desenvolupar.
- Escenaris d'ús: situacions concretes que ha de suportar l'aplicació.
- Recursos: descripció detallada de les classes de la biblioteca.
- Projecte base, persistència i instruccions de lliurament.

Important: La biblioteca gestiona automàticament la base de dades, la importació de GPX i els càlculs. L'alumne NO ha de modificar ni reimplementar aquestes classes, únicament utilitzar-les.

2. Cas d'estudi

El club Running la Safor vol desenvolupar una aplicació perquè els seus socis puguin registrar les seues activitats a l'aire lliure i fer-ne un seguiment i anàlisi. Després de registrar-se, els socis podran pujar les seues activitats a través de fitxers GPX generats per rellotges, dispositius wearables o aplicacions de tracking al mòbil.

Després de pujar el fitxer, l'aplicació permetrà visualitzar el traçat sobre un mapa, afegir anotacions geogràfiques, veure el perfil de desnivell i consultar estadístiques com ara distància, durada, velocitat i desnivell.

L'aplicació a desenvolupar també facilitarà certes tasques de configuració o de control que es detallen posteriorment.

A continuació es detallen els escenaris d'ús obtinguts de l'anàlisi de requisits del sistema, que cal utilitzar per dur a terme el disseny conceptual i després dissenyar i implementar adequadament l'aplicació requerida. Aquests escenaris s'han agrupat en diverses categories.

Categories d'escenaris

- Categoria 1 — Usuaris: registrar-se, autenticar-se, modificar perfil, tancar sessió, historial de sessions.
- Categoria 2 — Activitats: importar GPX, afegir anotacions, visualitzar activitat, acumulat, esborrar activitat.
- Categoria 3 — Ruta en mapa: zoom in/out, afegir anotacions.
- Categoria 4 — Altres: perfil de desnivell, velocitat sobre el traçat.
- Categoria 5 — Mapes: afegir un mapa al sistema.

3. Escenaris sobre l'usuari

3.1 Registrar-se a l'aplicació

En Joan vol registrar-se a l'aplicació. Després d'instal·lar-la, accedeix a l'opció de registre. El sistema sol·licita: nickname (identificador únic), adreça de correu electrònic, contrasenya, data de naixement i, opcionalment, un avatar.

En Joan introdueix el nickname "jgarcia", però el sistema l'informa que ja està en ús. Tria "jpgarcia". Introdueix la contrasenya "passPER21!" i el correu "jgarcia@gmail.com", amb data de naixement 14/03/2005. Decideix no pujar un avatar en aquest moment.

El sistema valida les dades d'acord amb les regles següents i registra l'usuari:

- Nickname: entre 6 i 15 caràcters, únicament lletres, dígit, guió o subguió.
- Contrasenya: entre 8 i 20 caràcters, amb almenys una majúscula, una minúscula, un dígit i un símbol (!@#\$%&*()-+=).
- Correu electrònic: format vàlid usuari@domini.
- Edat mínima: l'usuari ha de tenir més de 12 anys.

Nota d'implementació: Utilitza els mètodes estàtics `User.checkNickName()`, `User.checkEmail()`, `User.checkPassword()` i `User.isOlderThan()` per validar cada camp abans de cridar `app.registerUser()`.

3.2 Autenticar-se

En Joan obri l'aplicació i selecciona l'opció d'autenticació. El sistema sol·licita el nickname i la contrasenya. En Joan introdueix "jpgarcia" i la seua contrasenya. El sistema verifica les credencials i, si són correctes, concedeix accés a la resta de funcions.

3.3 Modificar perfil

En Joan decideix canviar el seu avatar per una foto del seu equip. Accedeix a l'opció de modificar perfil. El sistema mostra la informació actual: nickname (no editable), correu electrònic, contrasenya i data de naixement. En Joan selecciona una imatge de la seua màquina com a nou avatar i sol·licita guardar els canvis.

El sistema aplica les mateixes regles de validació que en el registre i, si tot és correcte, actualitza les dades.

Nota d'implementació: Utilitza `app.updateCurrentUser(email, password, birthDate, avatarPath)`. Si el camp de contrasenya es deixa en blanc, mantén la contrasenya actual (`user.getPassword()`) sense modificar-la.

3.4 Tancar sessió

En Joan decideix tancar l'aplicació. El sistema guarda automàticament les dades de la sessió: instant d'inici i fi, nombre d'activitats importades, nombre d'activitats visualitzades i nombre d'anotacions creades. Aquestes dades queden disponibles en l'historial.

3.5 Visualitzar l'historial de sessions

En Joan accedeix a l'historial de sessions per analitzar el seu ús de l'aplicació. El sistema mostra les sessions registrades amb la durada i les estadístiques d'ús (importades, vistes i anotacions creades), així com els totals acumulats.

4. Escenaris sobre les activitats

4.1 Registrar una activitat nova

En Joan selecciona un fitxer GPX d'una cursa del cap de setmana. El sistema el processa, mostra la ruta sobre el mapa ressaltant l'inici (verd) i el final (roig), i presenta les estadístiques: distància, durada, velocitat mitjana, ritme mitjà, desnivell positiu i negatiu, altitud mínima i màxima.

4.2 Afegir anotacions a l'activitat

En Joan recorda dues zones perilloses del recorregut i vol anotar-les. Selecciona l'activitat, fa clic dret sobre el mapa en el punt corresponent i tria el tipus d'anotació (punt, text, cercle o línia). Escriu el text i tria el color. Les anotacions queden persistides i es mostren en visualitzar l'activitat.

4.3 Visualitzar una activitat

En Joan selecciona una activitat de la llista. El sistema mostra el traçat sobre el mapa, les anotacions existents i les estadístiques calculades.

4.4 Acumulat d'activitats

En Joan consulta el total d'activitats del mes: temps total, distància acumulada, metres de pujada i baixada totals.

5. Escenaris sobre la ruta en el mapa

5.1 Realitzar zoom

En Joan amplia o redueix la vista del mapa mitjançant els botons de zoom (+/-). El traçat i les anotacions s'escalen amb el mapa.

6. Escenaris addicionals

6.1 Perfil de desnivell

Juntament amb el mapa es mostra una gràfica amb el perfil d'altitud de l'activitat (eix X: distància en km, eix Y: altitud en metres). En passar el ratolí sobre la gràfica, es ressaltava el punt corresponent sobre el mapa.

6.2 Velocitat sobre el traçat

El sistema permet visualitzar la velocitat seguida en cada tram del recorregut, codificada visualment sobre el traçat del mapa.

7. Escenaris sobre mapes

7.1 Afegir un mapa al sistema

En Joan ha participat en una cursa a Montanejos, però no té aquest mapa a l'aplicació. Accedeix a la gestió de mapes i introdueix el fitxer JPG del mapa juntament amb les coordenades geogràfiques del seu bounding box (latitud i longitud mínima i màxima). Aquestes coordenades les obté de l'script Python proporcionat, que genera la imatge i les imprimeix per pantalla.

Important: Els valors del bounding box han de coincidir exactament amb els que imprimeix l'script. Si s'utilitzen valors aproximats, la projecció de coordenades GPS a píxels serà incorrecta. L'script es facilita com a material addicional i s'executa fora de l'aplicació a desenvolupar.

8. Recursos per a realitzar la pràctica

Per centrar la pràctica en el disseny de la interfície i la interacció, es proporciona la biblioteca IPC2026.jar, que implementa el model de dades i proporciona tota la funcionalitat de suport. L'alumne ha d'utilitzar les classes ja implementades, sense reimplementar-les.

- Creació automàtica de la base de dades SQLite si no existeix (fitxer sportactivity.db en el directori del projecte).
- Emmagatzemament i recuperació d'usuaris, sessions, activitats, anotacions i mapes.
- Importació i anàlisi de fitxers GPX.

- Càlculs de distància, durada, desnivell i velocitat.
- Transformació de coordenades GPS a píxels de mapa.

8.1 Els mapes

L'estratègia per mostrar mapes consisteix a usar una imatge estàtica com a fons i dibuixar el traçat al damunt. Cada imatge té associades unes coordenades geogràfiques (bounding box) que permeten transformar posicions GPS en píxels.

En lloc d'una sola imatge, es disposa de diverses imatges i se n'escull l'adequada per a la ruta que es vol visualitzar. Cada mapa té associada una latitud i longitud mínima i màxima que permet transformar les posicions GPS en píxels del mapa.

La biblioteca inclou tres mapes per defecte amb les seues imatges i coordenades ja registrades:

Regió	Lat mín	Lat màx	Lon mín	Lon màx
Valencia	39.334297	39.554883	-0.505371	-0.285645
Sierra Calderona	39.588757	39.757880	-0.571289	-0.285645
Pirineos	42.585444	42.811522	0.000000	0.307617

Atenció: Aquests valors exactes són els que utilitza la biblioteca internament. Si afegiu mapes propis, useu sempre les coordenades que imprimeix l'script `generar_mapas_hd.py`, mai valors aproximats.

El codi bàsic per mostrar el mapa i la ruta en JavaFX és el següent:

```
// Carregar la imatge del mapa
Image map = new Image(new File(region.getImagePath()).toURI().toString());
ImageView mapView = new ImageView(map);

// Crear el Pane contenidor del mapa
Pane mapPane = new Pane(mapView);
mapPane.setPrefSize(map.getWidth(), map.getHeight());

// Crear la projecció per convertir GPS → píxels
MapProjection proj = new MapProjection(region, map.getWidth(), map.getHeight());

// Dibuixar la ruta
Polyline route = new Polyline();
for (TrackPoint tp : activity.getTrackPoints()) {
    Point2D p = proj.project(tp);
    route.getPoints().addAll(p.getX(), p.getY());
}
mapPane.getChildren().add(route);
```


8.2 Els fitxers GPX

Un fitxer GPX (GPS Exchange Format) és un estàndard obert basat en XML dissenyat per a l'intercanvi de dades GPS entre aplicacions i dispositius.

Un fitxer GPX sol contenir tres tipus de dades fonamentals: Waypoints, Routes i Tracks. Aquests últims són els que ens interessin en aquest projecte:

- Tracks: el registre real d'un recorregut ja realitzat, que inclou temps i elevació en cada punt.

Sol venir acompanyat de metadades com ara el nom, que també serà d'interès per al projecte.

Altres dades com la freqüència cardíaca o la temperatura no es tindran en compte en aquest projecte.

8.3 Model de dades i classes de la biblioteca

Després de l'estudi dels objectes de tasca (disseny conceptual) realitzat sobre els escenaris, s'han implementat les classes següents en el model:

- User
- Session
- Activity
- Annotation

A més de les classes obtingudes a partir dels escenaris, cal afegir les classes que permeten treballar amb el model de mapes i les dades GPS:

- TrackPoint
- GeoPoint
- MapRegion
- MapProjection
- GeoUtils
- SportActivityApp

Les classes del model es creen exclusivament per la biblioteca. L'alumne no les pot instanciar directament (els seus constructors són package-private). Només es poden instanciar directament:

- GeoPoint — per definir el punt geogràfic d'una anotació.
- Annotation — per crear noves anotacions abans d'afegir-les a l'activitat.
- MapProjection — per convertir coordenades en una vista de mapa.

Regla general: Totes les altres classes (Activity, TrackPoint, User, Session, MapRegion) s'obtenen exclusivament a través dels mètodes de SportActivityApp, mai amb new.

User

Emmagatzema la informació d'un usuari. El nickname no es pot modificar després del registre.

Mètode	Descripció
<code>getNickName()</code>	Retorna el nom d'usuari (no modificable).
<code>getEmail()</code> / <code>setEmail(email)</code>	Recupera o modifica l'adreça de correu electrònic.
<code>getPassword()</code> / <code>setPassword(pass)</code>	Recupera o modifica la contrasenya.
<code>getAvatarPath()</code> / <code>setAvatarPath(path)</code>	Recupera o modifica la ruta del fitxer d'avatar.
<code>getAvatar()</code>	Carrega i retorna l'avatar com a <code>javafx.scene.image.Image</code> . Retorna null si no hi ha avatar.
<code>getBirthDate()</code> / <code>setBirthDate(date)</code>	Recupera o modifica la data de naixement.
<code>getActivities()</code>	Retorna la llista d'activitats de l'usuari.
<code>getSessions()</code>	Retorna la llista de sessions de l'usuari.

Important — avatar: L'avatar s'emmagatzema com a ruta de fitxer (String `avatarPath`), NO com a objecte `Image`. Usa `getAvatarPath()` per guardar/recuperar la ruta, i `getAvatar()` per obtenir la `Image` quan la vulgues mostrar.

Mètodes estàtics de validació

Mètode	Descripció
<code>User.checkNickName(nick)</code>	True si el nickname compleix les regles (6-15 caràcters, lletres/dígits/guió/subguió).
<code>User.checkEmail(email)</code>	True si el correu té format vàlid.
<code>User.checkPassword(pass)</code>	True si la contrasenya compleix les regles (8-20 caràcters, majúscula, minúscula, dígit, símbol).
<code>User.isOlderThan(birthDate, years)</code>	True si la persona té estrictament més de 'years' anys.

Session

Representa una sessió d'ús. Es crea automàticament en tancar sessió. Els seus atributs no es poden modificar.

Mètode	Descripció
<code>getStartTime()</code> / <code>getEndTime()</code>	Instant d'inici i fi de la sessió.
<code>getDuration()</code>	Durada total de la sessió com a Duration.
<code>getImportedActivities()</code>	Nombre d'activitats importades durant la sessió.
<code>getViewedActivities()</code>	Nombre d'activitats visualitzades durant la sessió.
<code>getAnnotationsCreated()</code>	Nombre d'anotacions creades durant la sessió.

Activity

Representa una activitat esportiva importada des d'un fitxer GPX.

Mètode	Descripció
<code>getId()</code>	Identificador únic a la base de dades.
<code>getName()</code> / <code>setName(name)</code>	Recupera o modifica el nom. Per persistir el canvi usa <code>app.renameActivity()</code> .
<code>getUser()</code>	Propietari de l'activitat.
<code>getStartTime()</code> / <code>getEndTime()</code>	Data i hora d'inici i fi (del primer i últim TrackPoint).
<code>getTrackPoints()</code>	Llista no modificable de punts GPS en ordre de registre.
<code>getAnnotations()</code>	Llista no modificable d'anotacions associades.
<code>getSuggestedMap()</code>	MapRegion recomanada per representar l'activitat.
<code>getStartPoint()</code> / <code>getEndPoint()</code>	Primer i últim TrackPoint del recorregut.
<code>getTotalDistance()</code>	Distància total en metres (fórmula Haversine).
<code>getDuration()</code>	Durada total com a Duration.
<code>getAverageSpeed()</code>	Velocitat mitjana en km/h.
<code>getAveragePace()</code>	Ritme mitjà en minuts per kilòmetre.

<code>getElevationGain()</code> / <code>getElevationLoss()</code>	Desnivell positiu i negatiu acumulats en metres.
<code>getMinElevation()</code> / <code>getMaxElevation()</code>	Altitud mínima i màxima registrades en metres.

TrackPoint

Representa un punt del recorregut GPS emmagatzemat en el fitxer GPX.

Mètode	Descripció
<code>getLatitude()</code> / <code>getLongitude()</code>	Coordenades geogràfiques en graus decimals (WGS-84).
<code>getElevation()</code>	Altitud sobre el nivell del mar en metres.
<code>getTime()</code>	Instant UTC associat al punt. Pot ser null.
<code>distanceTo(other)</code>	Distància en metres fins a un altre TrackPoint (Haversine).
<code>timeTo(other)</code>	Diferència temporal entre dos punts com a Duration.
<code>speedTo(other)</code>	Velocitat mitjana entre dos punts en km/h.

Annotation

Representa una anotació de l'usuari sobre el mapa. És l'única classe del model que l'alumne pot instanciar directament (sense id):

```
// Constructor públic (sense id — la biblioteca l'assigna en guardar):
Annotation ann = new Annotation(
    AnnotationType.POINT, // tipus d'anotació
    "Zona peligrosa",     // text (pot ser buit)
    "#E74C3C",           // color en format CSS hex
    2.0,                  // grossor del traç
    List.of(geoPoint)     // punts geogràfics (vegeu taula de tipus)
);
// Persistir i obtenir l'anotació amb l'id assignat:
Annotation saved = app.addAnnotation(activity, ann);
```

Mètode	Descripció
<code>getId()</code>	Identificador a la BD (0 si encara no s'ha persistit).
<code>getType()</code>	Tipus d'anotació (AnnotationType).

<code>getText()</code> / <code>setText(text)</code>	Text associat a l'anotació.
<code>getColor()</code> / <code>setColor(color)</code>	Color en format CSS hex, p. ex. "#E74C3C".
<code>getStrokeWidth()</code> / <code>setStrokeWidth(w)</code>	Grossor del traç en píxels.
<code>getGeoPoints()</code>	Llista no modificable de punts geogràfics.

AnnotationType

Enumeració amb els tipus d'anotació disponibles. Cada tipus requereix un nombre específic de GeoPoints:

Valor	Descripció	GeoPoints necessaris
POINT	Cercle ple amb etiqueta de text opcional.	1 (posició)
TEXT	Etiqueta de text ancorada a una posició.	1 (ancla)
LINE	Segment recte entre dos punts.	2 (inici, fi)
CIRCLE	Circumferència definida per centre i bora.	2 (centre, bora)

GeoPoint

Coordenada geogràfica immutable. Un dels tres tipus que l'alumne pot instanciar directament.

```
// Constructor públic:
GeoPoint punto = new GeoPoint(latitud, longitud);

// Obtener a partir d'un clic en el mapa:
GeoPoint geo = projection.unproject(e.getX(), e.getY());
```

Mètode	Descripció
<code>getLatitude()</code>	Latitud en graus decimals.
<code>getLongitude()</code>	Longitud en graus decimals.

MapRegion

Representa una regió geogràfica associada a una imatge de mapa. Només es pot obtenir a través de SportActivityApp (constructor package-private).

Mètode	Descripció
<code>getName()</code>	Nom de la regió, p. ex. "Valencia".
<code>getImagePath()</code>	Ruta relativa a la imatge, p. ex. "maps/valencia.jpg".
<code>getLatMin()</code> / <code>getLatMax()</code>	Límits sud i nord del bounding box en graus decimals.
<code>getLonMin()</code> / <code>getLonMax()</code>	Límits oest i est del bounding box en graus decimals.
<code>contains(lat, lon)</code>	True si la coordenada és dins del bounding box.
<code>contains(activity)</code>	True si tots els punts de l'activitat estan dins del bounding box.

MapProjection

Converteix entre coordenades geogràfiques i píxels d'imatge usant la projecció Web Mercator. Creeu una instància per cada vista de mapa activa.

```
// Constructor (l'alumne pot instanciar directament):
MapProjection proj = new MapProjection(region, imgWidth, imgHeight);
```

Mètode	Descripció
<code>project(lat, lon)</code>	Converteix coordenada geogràfica → Point2D (píxels a la imatge).
<code>project(GeoPoint p)</code>	Versió per a GeoPoint.
<code>project(TrackPoint p)</code>	Versió per a TrackPoint.
<code>projectActivity(activity)</code>	Converteix tots els TrackPoints d'una activitat → List<Point2D>.
<code>unproject(screenX, screenY)</code>	Converteix píxels del Pane → GeoPoint. Passa <code>e.getX()/e.getY()</code> directament; JavaFX ja corregeix el zoom.
<code>getRegion()</code>	Retorna la MapRegion associada a aquesta projecció.

Ús d'unproject(): Crida `projection.unproject(e.getX(), e.getY())` per transformar les coordenades del ratolí. JavaFX corregeix automàticament les coordenades segons el nivell de zoom, de manera que no cal escalar manualment.

GeoUtils

Classe d'utilitat amb mètodes estàtics per a càlculs geogràfics. Tots els càlculs de distància usen la fórmula Haversine (error < 0,5% per a distàncies típiques d'esport).

Mètode	Descripció
<code>GeoUtils.distance(lat1, lon1, lat2, lon2)</code>	Distància entre dues coordenades en metres.
<code>GeoUtils.distance(GeoPoint, GeoPoint)</code>	Distància entre dos GeoPoints en metres.
<code>GeoUtils.distance(TrackPoint, TrackPoint)</code>	Distància entre dos TrackPoints en metres.
<code>GeoUtils.totalDistance(List<TrackPoint>)</code>	Distància total del recorregut en metres.
<code>GeoUtils.elevationGain(List<TrackPoint>)</code>	Desnivell positiu acumulat en metres.
<code>GeoUtils.elevationLoss(List<TrackPoint>)</code>	Desnivell negatiu acumulat en metres (valor positiu).
<code>GeoUtils.bearing(GeoPoint, GeoPoint)</code>	Rumb des de p1 fins a p2 en graus (0-360, sentit horari des del nord).

8.4 La classe SportActivityApp

Aquesta és la classe principal de la biblioteca. Implementa el patró Singleton: només existeix una instància durant l'execució. Agrupa tots els mètodes necessaris per gestionar usuaris, activitats, anotacions, mapes i sessions.

```
// Obtener la instància (sempre així, mai amb new):
SportActivityApp app = SportActivityApp.getInstance();
```

Gestió d'usuaris

Mètode	Descripció
<code>registerUser(nick, email, pass, birthDate, avatarPath)</code>	Registra un nou usuari. Valida tots els camps. Retorna true si té èxit.
<code>login(nickName, password)</code>	Autentica un usuari i inicia la sessió. Retorna true si les credencials són correctes.
<code>logout()</code>	Tanca la sessió activa i guarda automàticament les estadístiques de sessió a la BD.

<code>getCurrentUser()</code>	Retorna l'User autenticat actualment, o null si no hi ha sessió.
<code>updateCurrentUser(email, pass, birthDate, avatarPath)</code>	Actualitza el perfil de l'usuari autenticat. Valida tots els camps.

Gestió d'activitats

Mètode	Descripció
<code>importActivity(File gpxFile)</code>	Analitza un fitxer GPX, emmagatzema l'activitat a la BD i retorna l'objecte Activity creat.
<code>getUserActivities()</code>	Retorna les activitats de l'usuari autenticat, ordenades per data descendent.
<code>getActivitiesByUser(User)</code>	Retorna les activitats de qualsevol usuari.
<code>removeActivity(Activity)</code>	Elimina una activitat i totes les seues dades de la BD.
<code>renameActivity(Activity, String)</code>	Canvia el nom d'una activitat i el persisteix.

Gestió d'anotacions

Mètode	Descripció
<code>addAnnotation(Activity, Annotation)</code>	Guarda una anotació a la BD. Retorna l'Annotation amb l'id assignat, o null si falla.
<code>removeAnnotation(Annotation)</code>	Elimina una anotació de la BD.
<code>updateAnnotationColor(Annotation)</code>	Persisteix el canvi de color d'una anotació existent.

Gestió de mapes

Mètode	Descripció
<code>getMapRegions()</code>	Retorna totes les regions de mapa registrades, ordenades per nom.
<code>getUnusedMapRegions()</code>	Retorna només les regions que no estan referenciades per cap activitat (candidates a esborrar).
<code>findMapForActivity(Activity)</code>	Troba la regió que millor cobreix el traçat de l'activitat.

<code>findMapForPoint(lat, lon)</code>	Troba la regió que conté una coordenada geogràfica.
<code>addMapRegion(name, imageFile, latMin, latMax, lonMin, lonMax)</code>	Registra una nova regió: copia la imatge a maps/ i insereix el registre. Retorna la MapRegion creada o null si falla.
<code>removeMapRegion(MapRegion)</code>	Esborra una regió si cap activitat la referencia. També esborra el fitxer d'imatge (excepte els tres mapes per defecte).

Sessions

Mètode	Descripció
<code>getSessionsByUser(User)</code>	Retorna les sessions de l'usuari, ordenades per data descendent.

9. Exemple d'ús i estructura del projecte

9.1 Exemple d'ús

```

SportActivityApp app = SportActivityApp.getInstance();

// 1. Autenticació
boolean ok = app.login("jpgarcia", "passPER21!");

if (ok) {
    // 2. Importar activitat des d'un fitxer GPX
    Activity activity = app.importActivity(new File("ruta.gpx"));

    // 3. Obtenir el mapa adequat i crear la projecció
    MapRegion region = activity.getSuggestedMap();
    Image img = new Image(new File(region.getImagePath()).toURI().toString());
    MapProjection proj = new MapProjection(region, img.getWidth(),
    img.getHeight());

    // 4. Dibuixar el traçat
    Polyline route = new Polyline();
    for (TrackPoint tp : activity.getTrackPoints()) {
        Point2D p = proj.project(tp);
        route.getPoints().addAll(p.getX(), p.getY());
    }
    mapPane.getChildren().add(route);

    // 5. Mostrar estadístiques
    System.out.printf("Distància: %.2f km\n",
    activity.getTotalDistance()/1000.0);
    System.out.printf("Durada: %s\n", activity.getDuration());
    System.out.printf("Desnivell+: %.0f m\n", activity.getElevationGain());

    // 6. Tancar sessió (guarda estadístiques automàticament)
    app.logout();

```

```
}
```

9.2 Estructura del projecte a desenvolupar

El directori de treball de NetBeans és la carpeta arrel del projecte. La biblioteca crea i busca tots els seus fitxers relatius a eixa carpeta. L'estructura següent mostra com s'han d'organitzar els recursos necessaris perquè l'aplicació funcione correctament.

```
MiProyecto/      <- arrel del projecte NetBeans
├── lib/          <- biblioteques externes
│   ├── IPC2026.jar      <- biblioteca proporcionada
│   └── sqlite-jdbc-x.x.x.jar <- SQLite JDBC (descarregar de Maven)
├── maps/         <- imatges de mapa
│   ├── valencia.jpg    <- copiar abans d'executar
│   ├── calderona.jpg    <- copiar abans d'executar
│   └── pirineos.jpg     <- copiar abans d'executar
├── avatars/      <- creada automàticament en registrar
└── usuarios
    └── sportactivity.db <- creada automàticament en arrencar
```

Important: La carpeta avatars/ i el fitxer sportactivity.db es creen automàticament la primera vegada que s'executa l'aplicació. La carpeta maps/ i els tres fitxers JPG s'han de copiar manualment abans de la primera execució. Si els mapes no hi són, la biblioteca registra les regions a la base de dades però no pot mostrar les imatges.

9.3 Afegir les biblioteques a NetBeans

Per afegir IPC2026.jar i sqlite-jdbc al projecte:

1. Clic dret sobre el projecte → Properties → Libraries
2. A la pestanya Compile, prémer 'Add JAR/Folder'
3. Seleccionar lib/IPC2026.jar
4. Repetir per a lib/sqlite-jdbc-x.x.x.jar

La biblioteca SQLite JDBC es pot descarregar des de polifomat o des de <https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc>. Descarregueu el fitxer .jar i guardeu-lo a la carpeta lib/ del projecte.

Versió SQLite: Useu la versió 3.45 o superior. La biblioteca IPC2026.jar ha estat compilada i provada amb sqlite-jdbc-3.45.x.jar.

10. Persistència de les dades

La biblioteca utilitza una base de dades SQLite que es crea automàticament en el directori de treball del projecte amb el nom `sportactivity.db`. No cal configurar res; la biblioteca inicialitza les taules en arrencar.

Si necessiteu inspeccionar el contingut de la base de dades durant el desenvolupament, podeu usar l'aplicació gratuïta DB Browser for SQLite (<https://sqlitebrowser.org>).

Es proporcionarà una base de dades amb un usuari genèric per poder provar l'aplicació abans d'haver implementat el registre d'usuaris.

11. Projecte base

Es proporciona el projecte NetBeans MapaDemo com a punt de partida. Aquest projecte mostra com implementar el zoom sobre el mapa usant un Group dins d'un ScrollPane, que és l'estructura necessària perquè l'escalat funcione correctament amb els nodes superposats.

Recomanació: Useu aquest projecte com a base, eliminant o adaptant allò que no siga necessari per a la vostra implementació. L'estratègia adequada passa per generar cada vista en un fitxer fxml diferent i anar canviant les diferents àrees del BorderPane segons les accions de l'usuari.

12. Instruccions de lliurament

El lliurament es realitzarà en grups de 4 alumnes del mateix grup de laboratori. El procés de lliurament és el següent:

- Realitzeu el disseny conceptual i els prototips de baixa fidelitat abans d'implementar.
- Reviseu els prototips amb el vostre professor de pràctiques o de teoria.
- Creeu un compte a GitHub i pugeu el codi i la documentació de forma incremental.
- GitHub registra quins canvis ha realitzat cada integrant — això forma part de l'avaluació.
- El lliurament final es realitza a través d'una tasca de poliformat.

Important: Només s'acceptarà un grup amb menys de 4 integrants per grup de laboratori, en cas que no hi haja alumnes suficients per completar tots els grups.